2026.08.21

STL /CAD → RAW GEOMETRY

Processing Pipeline Implementation

Chloe Yu (Seojin Yu)
Research Intern, ESLAB, Seoul National University
Emory University, B.S. in CS and Mathematics

Agenda

| 01

Background

| 02

Preliminary Study

| 03

Pipeline Implementation

| 04

Engineering Challenges

| 05

CAD (STEP) Format Support

| 06

Solver Integration

| 07

Pipeline Extension

| 08

Multi-Building Domain

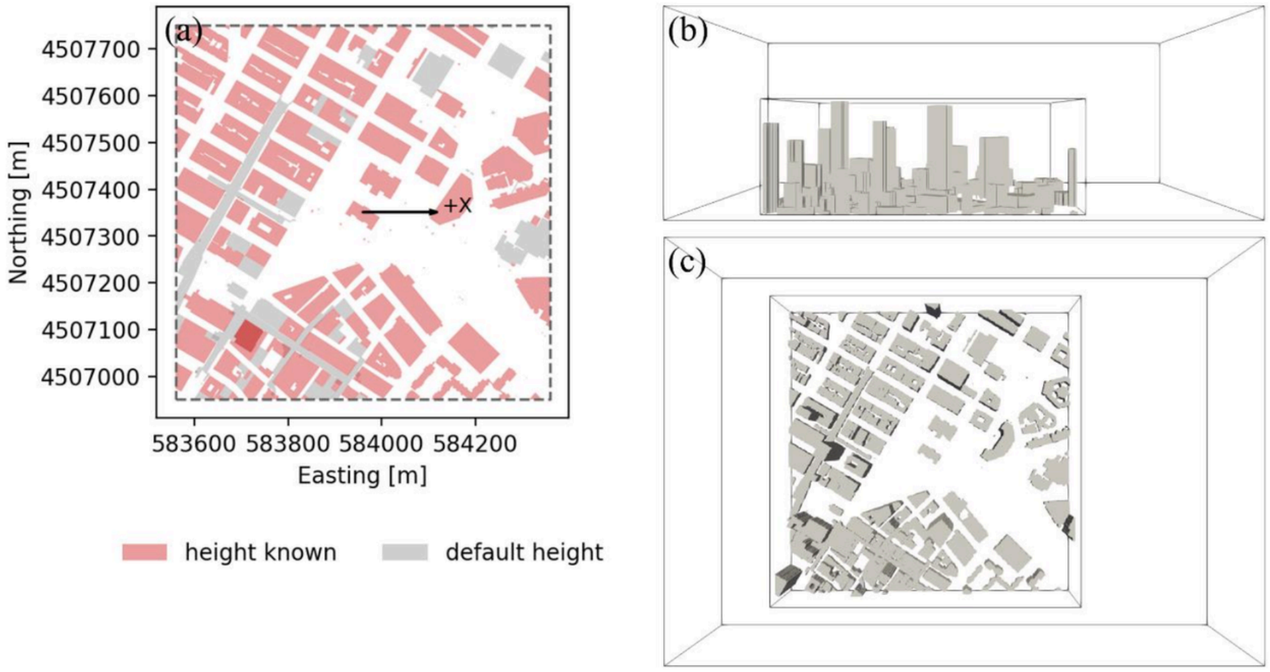
1.

BACKGROUND

Objectives and Rationale

- LBM solver requires fluid (air) / solid (building) classification at every grid cell
- Existing input path: OpenStreetMap (OSM)
  - footprint + height/floor count → grid placement
- Limitations of OSM:
  - Missing height data is replaced with a default value
  - Cannot represent complex curved surfaces or roof structures
  - Shapes not present on the map cannot be input at all

Objectives and Rationale



Extraction of building footprints from OpenStreetMap and voxelization into 3D urban geometry — the input geometry that determines simulation accuracy

Figure 2: Generation of the real-scale Manhattan urban geometry: (a) building footprints extracted from OpenStreetMap data, (b) side view of the voxelized urban geometry, and (c) top view of the voxelized urban geometry

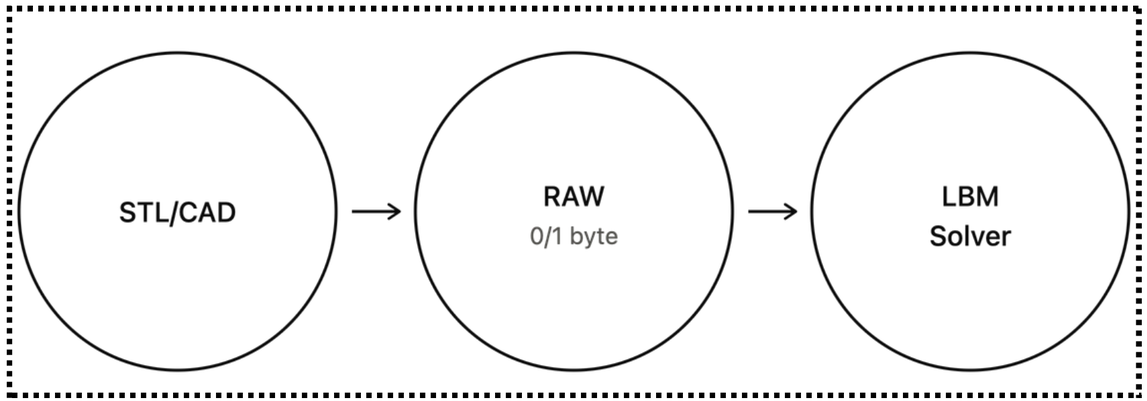
STL/CAD Input Path + Raw Format

Format

Goal: Not to replace OSM, but to supplement custom shapes OSM cannot handle, using STL/CAD

Why use the RAW:

- The solver doesn't need to parse STL every run
- Continuous bytes are read directly via C's fread
- Can be copied straight into GPU memory
- Converted once during preprocessing — the solver stays format-agnostic



STL/CAD Input Path + Raw Format

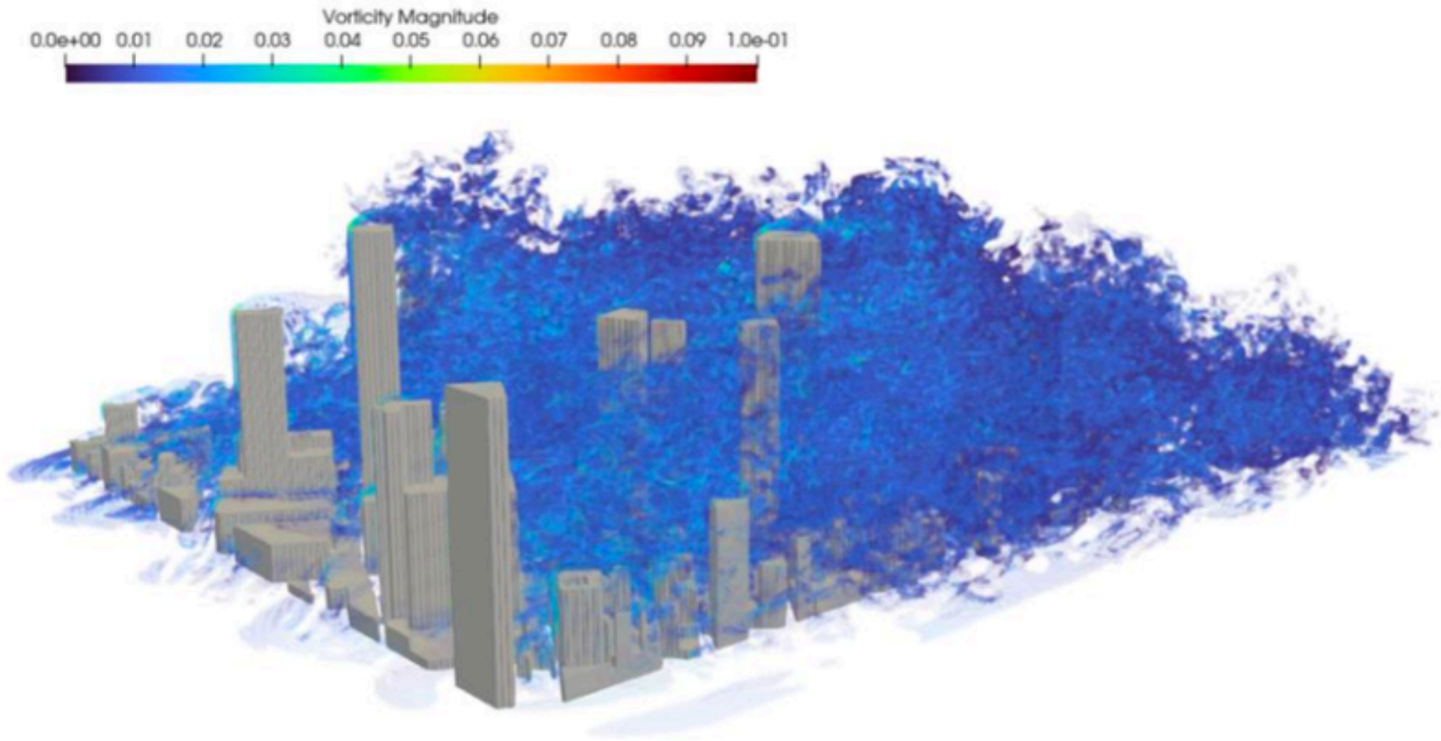


Figure 1: Three-dimensional visualization of the vorticity magnitude (in lattice units) in the real-scale Manhattan urban simulation

Vorticity distribution from a downtown Manhattan simulation — accurate building geometry input is required to obtain results like this

2.

PRELIMINARY STUDY

LBM Theory · Solver Implementation · Reynolds Sweep

Theory study:

Boltzmann equation, BGK approximation, D3Q19 discretization model

Solver implementation:

Implemented a 3D lid-driven cavity solver (60^3 grid, $\tau=0.7$), validated against the low-Re vortex pattern; a Re sweep (420–2000) showed n=60 diverging throughout, motivating the grid-resolution study.

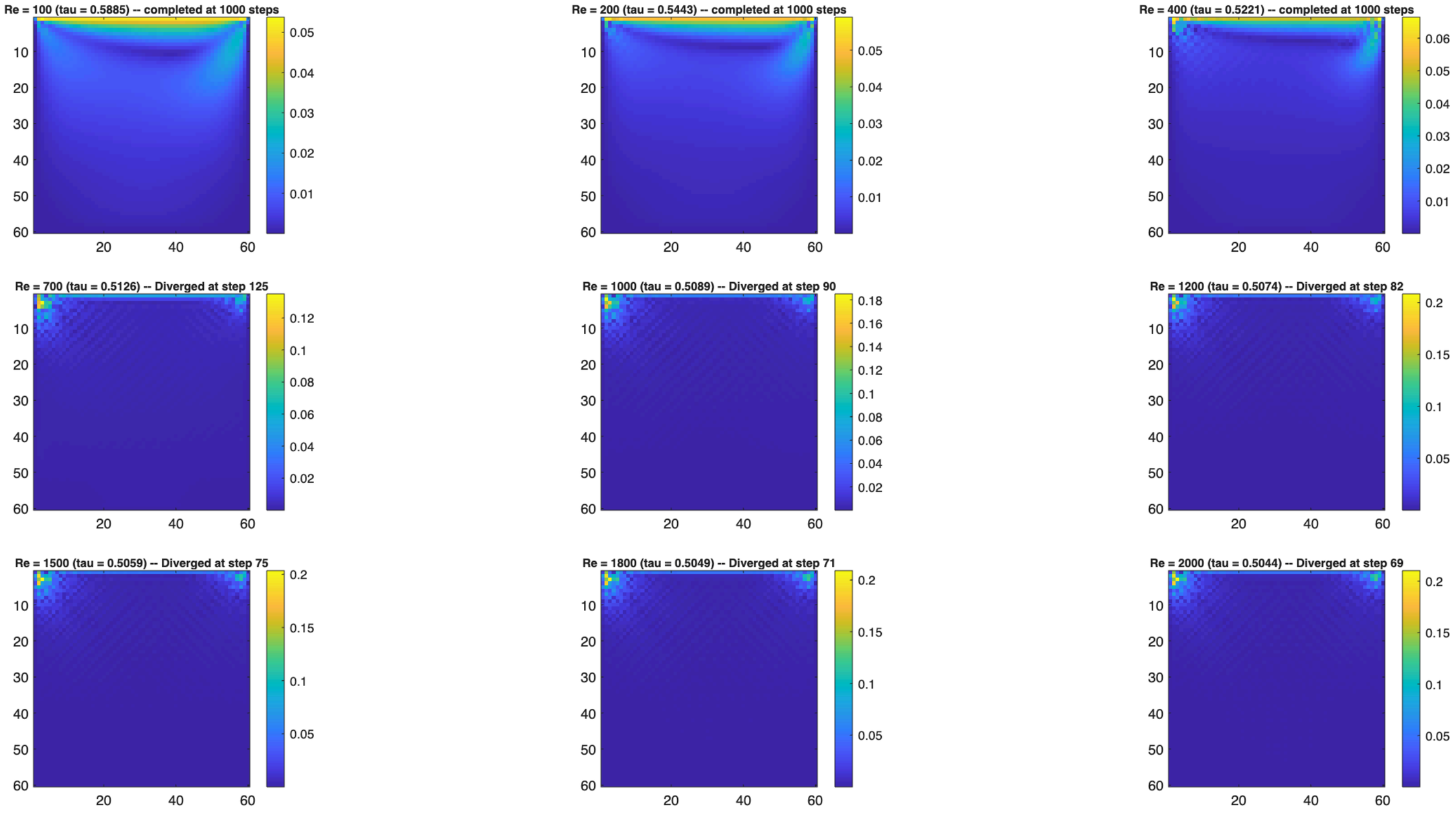
Reynolds sweep:

60^3 diverged → 100^3/128^3 stable; confirmed grid resolution issue

Significance:

A foundational learning stage before full pipeline development — established the basis for later grid definition and validation strategy

Re Sweep Results (60^3 cavity, y=mid plane)



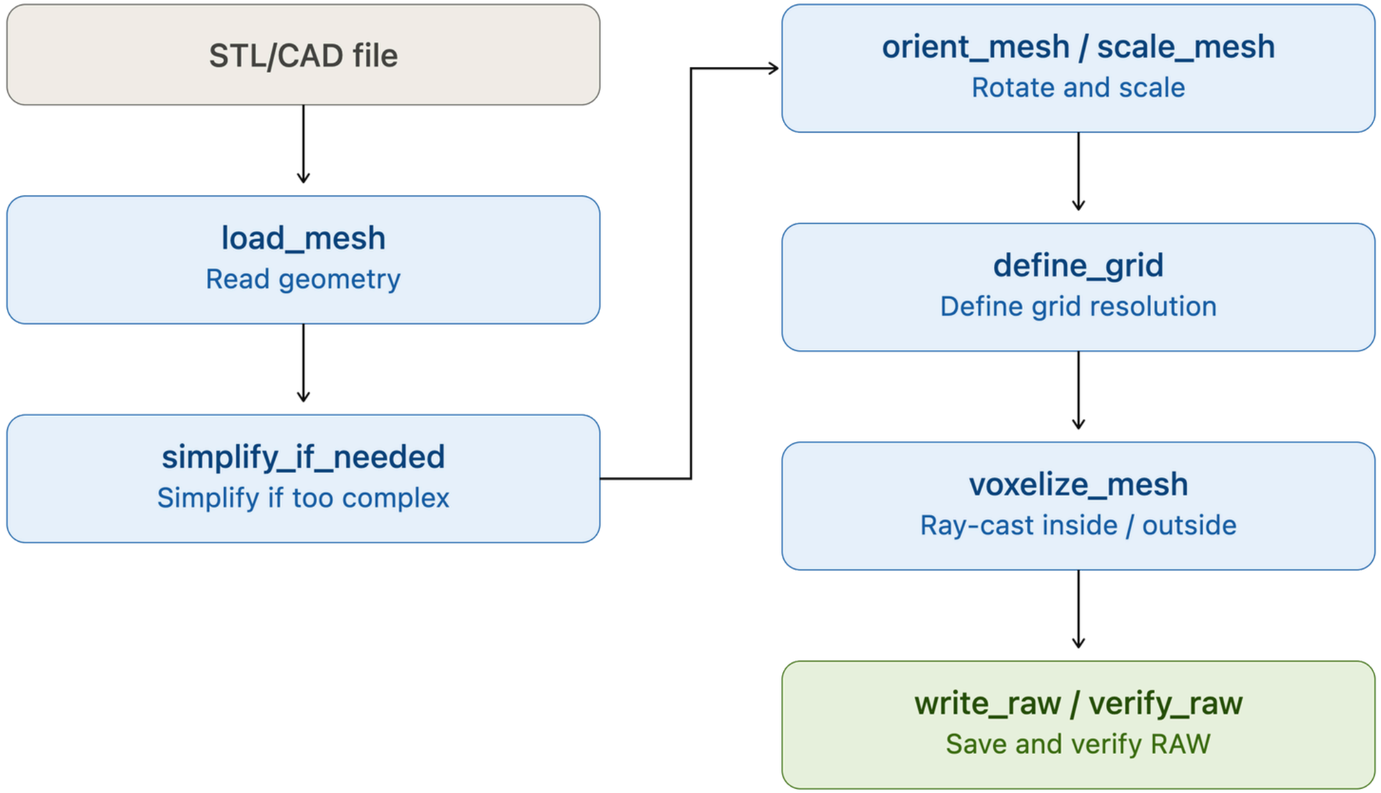
3.

STL/CAD → RAW PIPELINE:

RAW File Specification and Processing Flow

Data Type	uint8
Values	0 = fluid / 1 = solid
Storage order	C-order, Z-fastest
Header	none

$$index = z + nz \times (y + ny \times x)$$

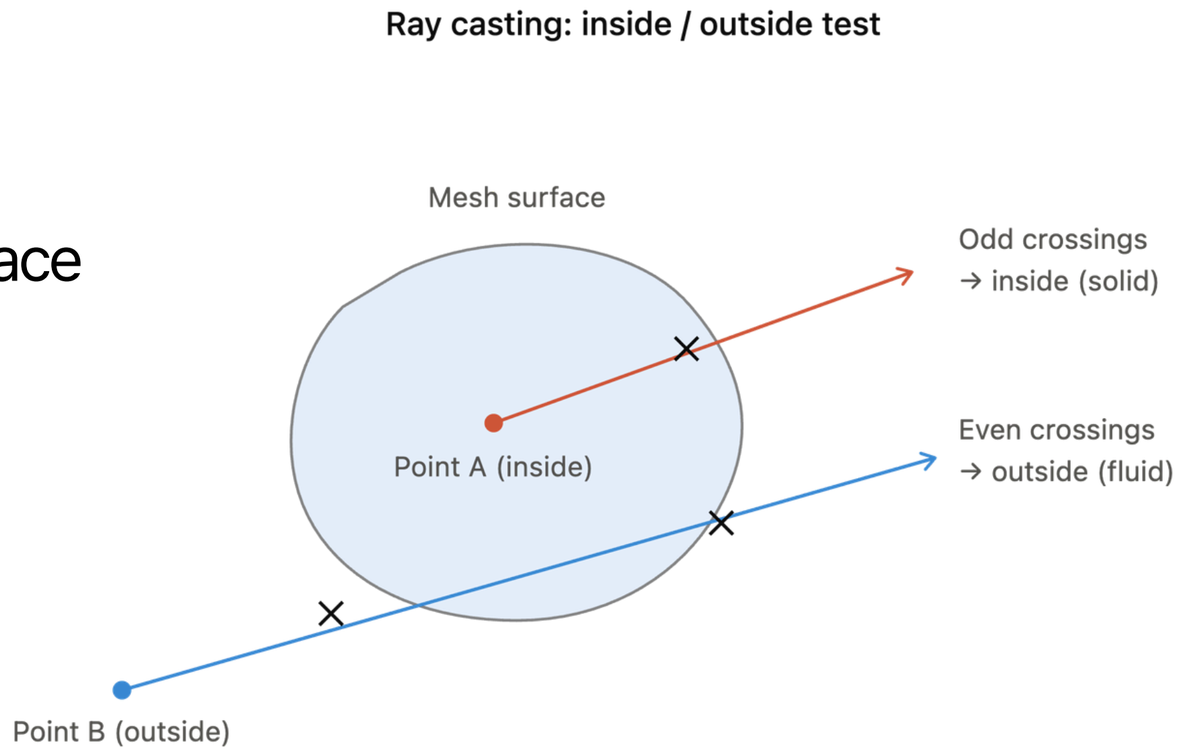


```python
def write_raw(occupancy, output_path):
    # no. 5:
    #   자료형: uint8, 유체=0, 고체=1, header 없음
    #   가장 빠르게 변하는 축: Z (index = z + nz*(y + ny*x))
    # numpy 배열이 (nx, ny, nz) shape이면 .tofile()의 기본 저장 순서
    # (C-order)가 이 규칙과 그대로 일치함.
    assert occupancy.dtype == np.uint8, "occupancy must be uint8"
    occupancy.tofile(output_path)

    print("=" * 50)
    print("STAGE 4: WRITE RAW")
    print("=" * 50)
    print(f"wrote {output_path} ({occupancy.size:,} bytes expected)")
```

Ray Casting Principle

- Cast a ray from each grid cell's center
- Count intersections with the mesh surface
- **Odd number → inside (solid)**
- **Even number → outside (fluid)**
- Assumes a watertight (closed) shape



```python
def voxelize_mesh(mesh, nx, ny, nz, grid_info):
    x_centers = grid_info["x_centers"]
    y_centers = grid_info["y_centers"]
    z_centers = grid_info["z_centers"]

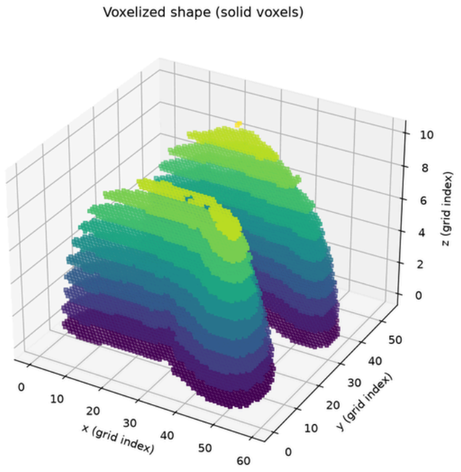
    xx, yy, zz = np.meshgrid(x_centers, y_centers, z_centers, indexing="ij")
    points = np.stack([xx.ravel(), yy.ravel(), zz.ravel()], axis=-1)

    print(f"testing {points.shape[0]:,} points against the mesh...")
    inside = mesh.contains(points)

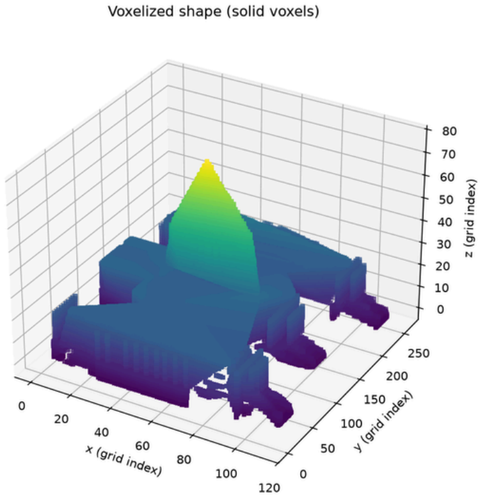
    # no.3: "유체는 0, 고체는 1로 표시"
    occupancy = inside.reshape(nx, ny, nz).astype(np.uint8)
    return occupancy
```

VALIDATION

01	02	03	04	05	06
synthetic shapes	byte order	40MB real file	NASA Eros	Capitol Building	C re-verification



NASA Eros asteroid
(35.87% solid)



U.S. Capitol Building

(CC BY-NC-SA 4.0)

Independent re-verification in C

Python and C results matched exactly

in solid voxel count

313,994

voxels — Python = C

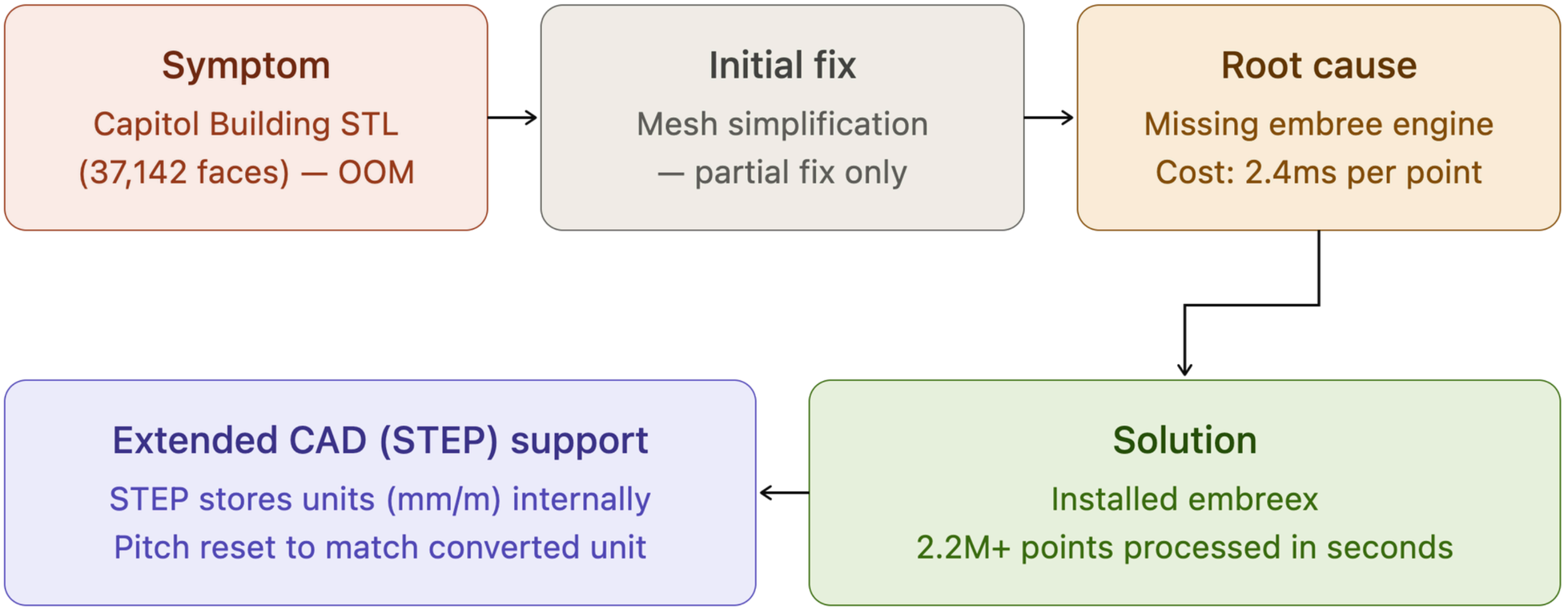
```
STAGE 3: VOXELIZATION RESULT
==================================================
shape: (114, 265, 78), dtype: uint8
solid voxels: 313,994 / 2,356,380 (13.33%)

(venv) chloe@yuseojins-MacBook-Air geometry_pipeline % ./read_raw capitol_fine.raw
114 265 78
expected bytes : 2356380
actual bytes   : 2356380
values are 0/1 only : OK
solid voxels        : 313994 / 2356380 (13.33%)
data[0] (x=0,y=0,z=0) = 0
data[1] (x=0,y=0,z=1) = 0
```

4.

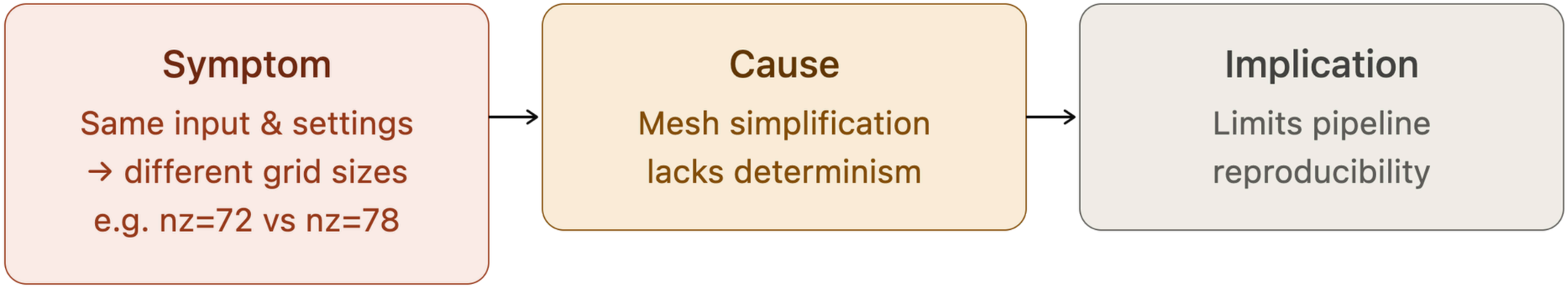
REAL-WORLD ENGINEERING PROBLEM SOLVING

Out-of-Memory(OOM) Debugging



Reproducibility Issue

Non-Watertight Meshes



5.

CAD (STEP) FORMAT SUPPORT

Extending CAD Support to STEP Files

Extended trimesh via cascadio

Added the cascadio library so trimesh can parse STEP files directly, extending the pipeline beyond STL/mesh input.

→

Validated with cadquery

Generated an actual STEP file (a box + a cylinder) using cadquery, then ran it through the full pipeline to confirm correct parsing.

→

Unit handling discovered

STEP stores an explicit unit (mm/m) in the file, which trimesh auto-converts. Unlike STL, pitch must be reset relative to the converted unit.

This closes a gap in the STL/CAD input path: designs authored natively in CAD tools (STEP) can now feed the same RAW pipeline as STL meshes.

Key finding

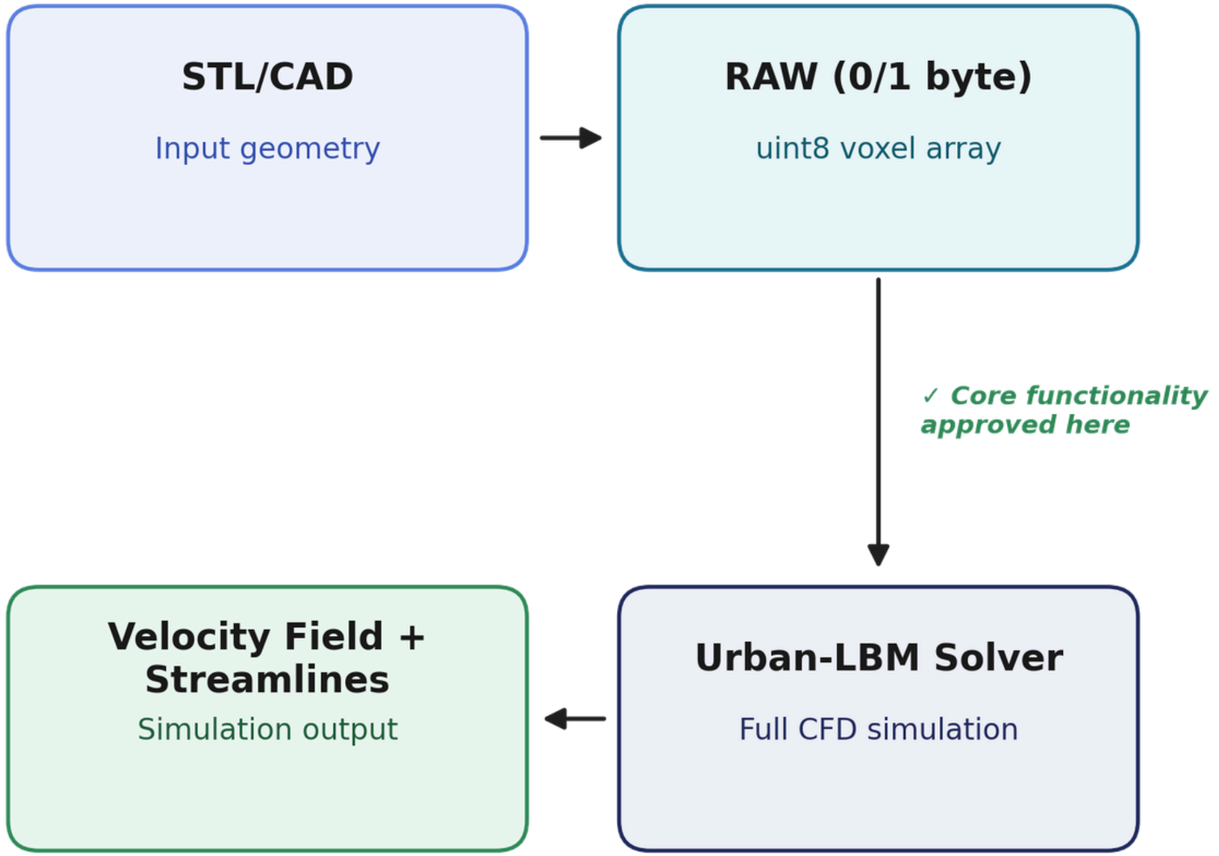
A STEP file carries its own unit definition (mm or m), and trimesh converts it automatically on load. STL has no such concept — so the grid pitch must be re-derived against the post-conversion unit, not assumed to already be meters.

6.

SOLVER INTEGRATION VALIDATION

First Validation — Pipeline Approved

Fed the generated RAW file into the actual Urban-LBM solver to run a simulation (velocity distribution, streamlines)



Second Validation
— Large-Scale Production Simulation

112.5M
voxel grid
(900 x 500 x 250)

1.71×
Real-time factor
(faster than real time)

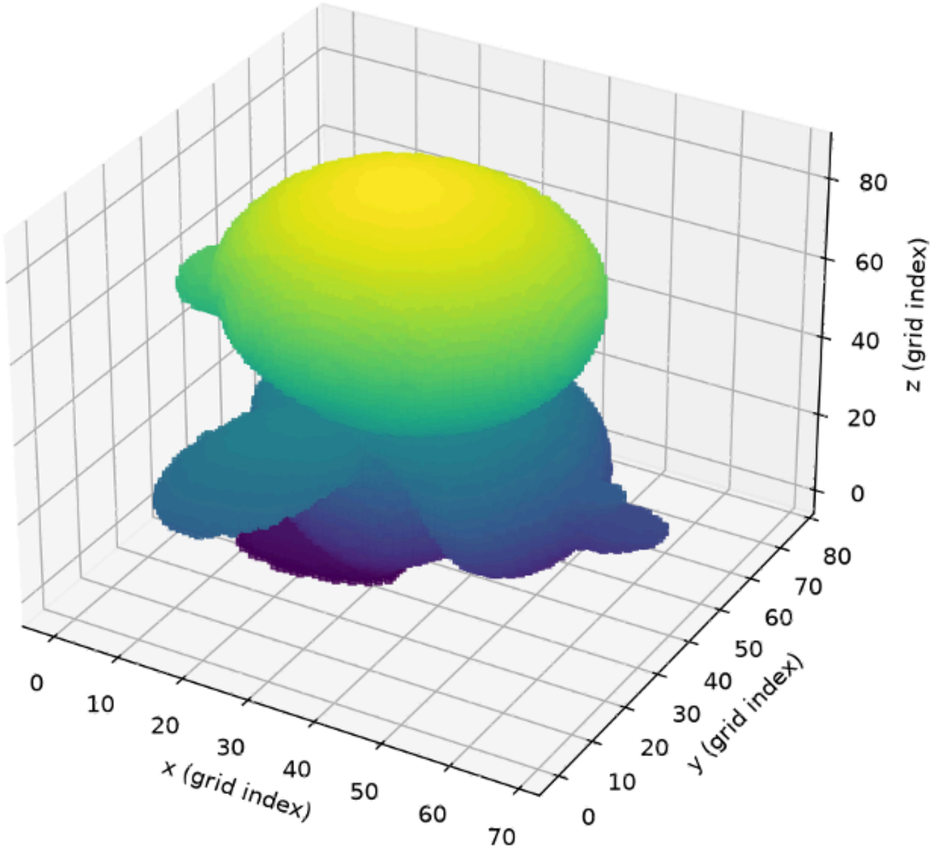
1,800m
Domain size
(x-direction)

Test shapes: Piplup (팽도리) , Psyduck (고라파덕)

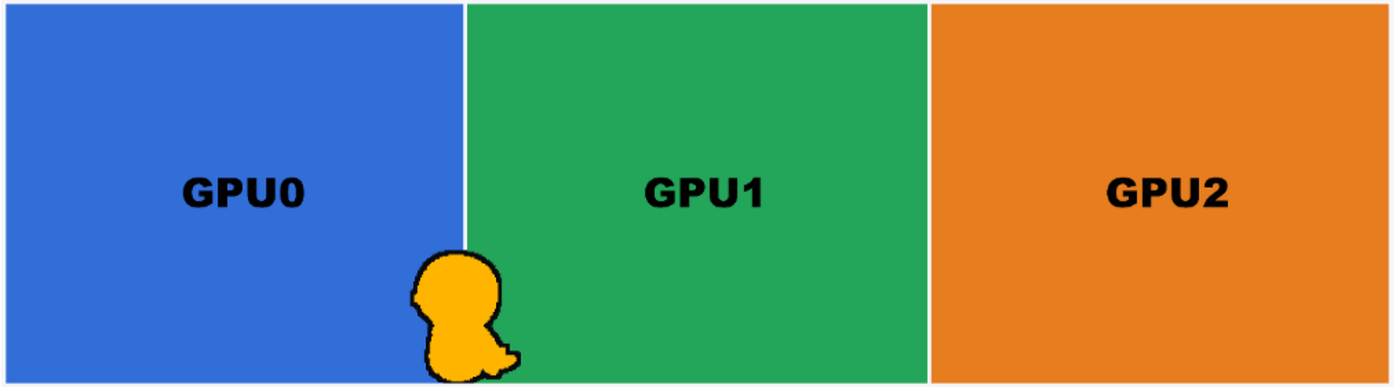
Item	Case 1 · Piplup	Case 2 · Psyduck
Shape height	170 m	160 m
Solid cell count	125,082	85,328
Orientation	Facing -x (inlet side)	Facing -x (inlet side)

Note: A case where the --target-height, --scale-axis, and --facing options were applied directly to an actual large-scale production simulation

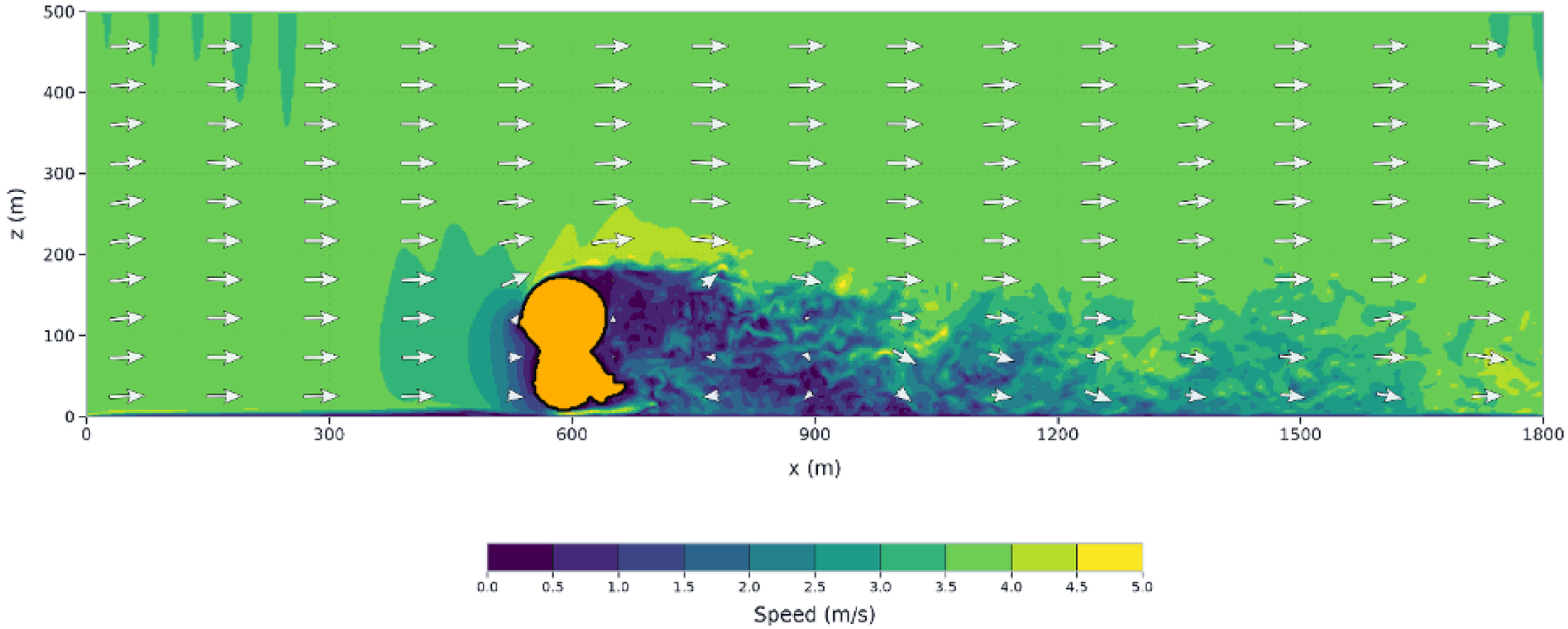
Velocity Field · Streamlines



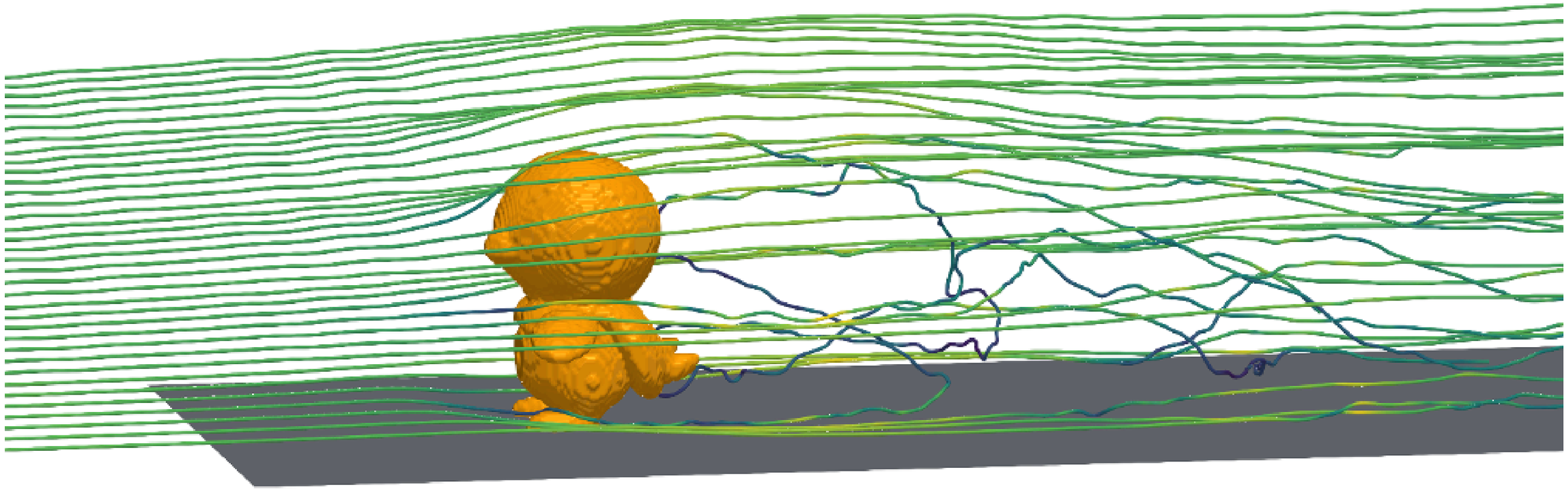
Piplup 팽도리 voxelize result
(STL to RAW conversion check)



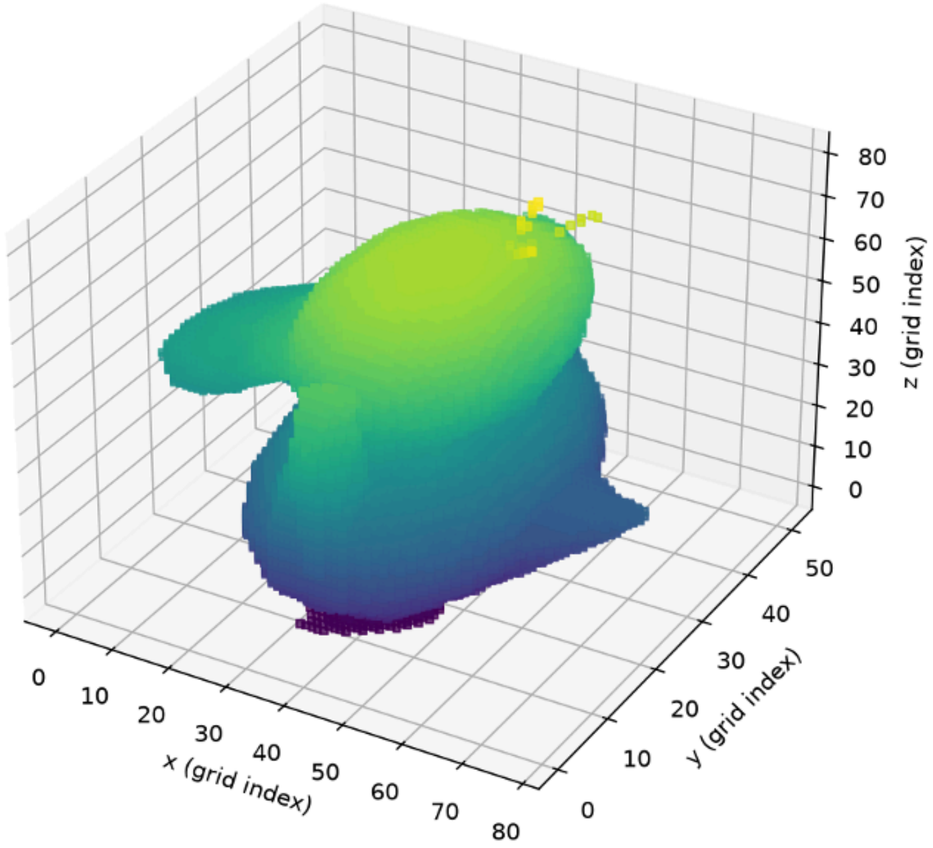
Piplup 팽도리 GPU domain-split visualization
(multi-GPU parallel processing context)



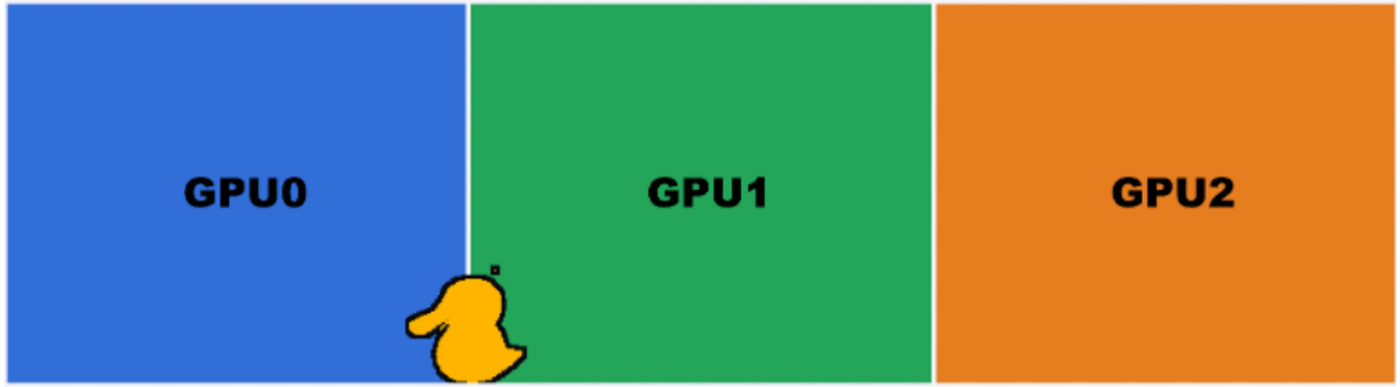
Piplup 팽도리 simulated wind-tunnel result (y = Ly/2 cross-section)
— y-plane velocity distribution + vector visualization



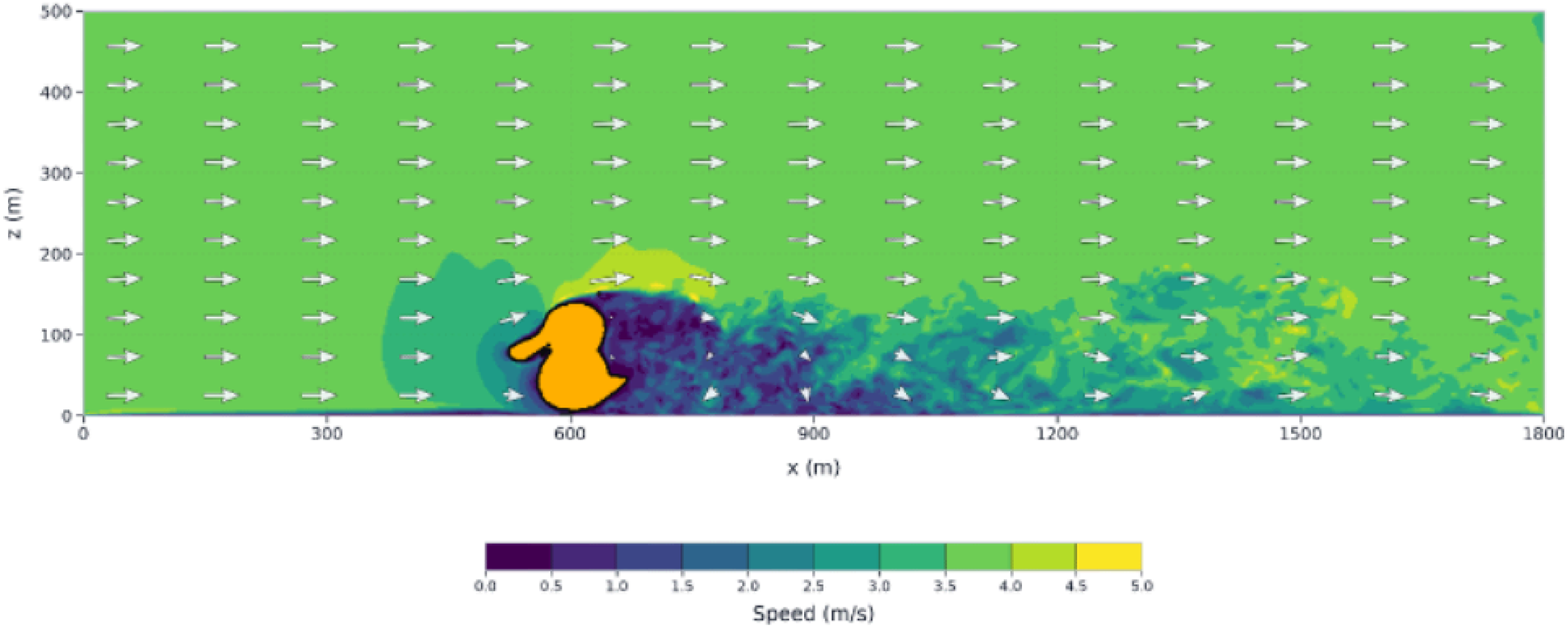
Piplup 팽도리 simulated wind-tunnel result (3D streamlines)
— 3D streamline visualization



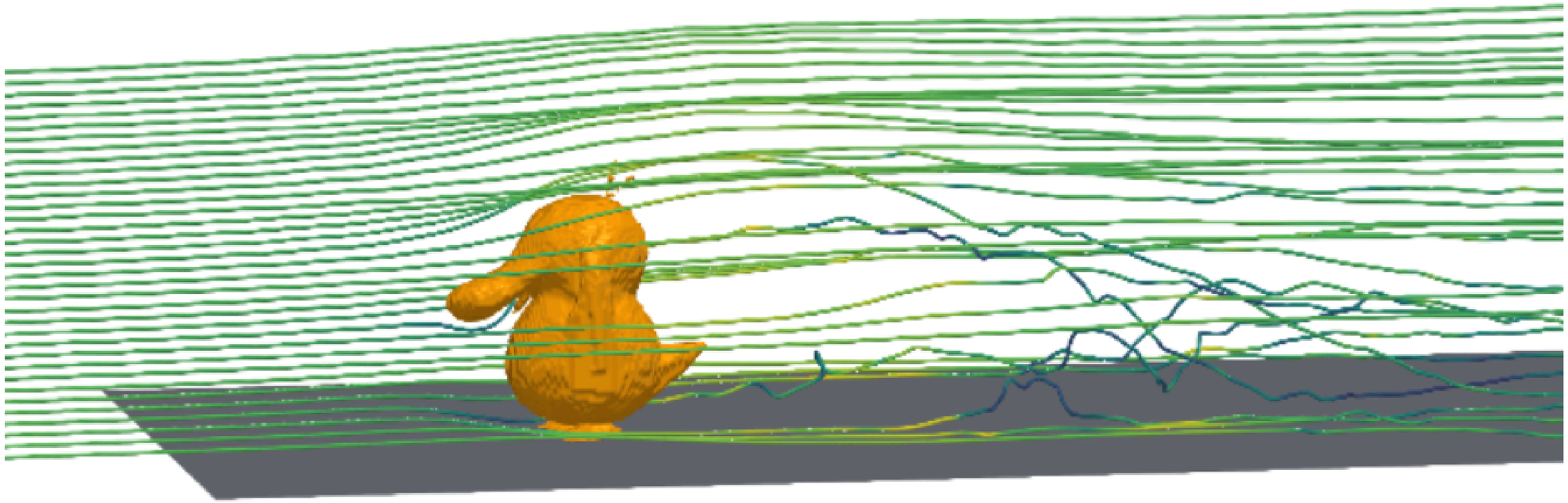
Psyduck 고라파덕 voxelize result
(STL to RAW conversion check)



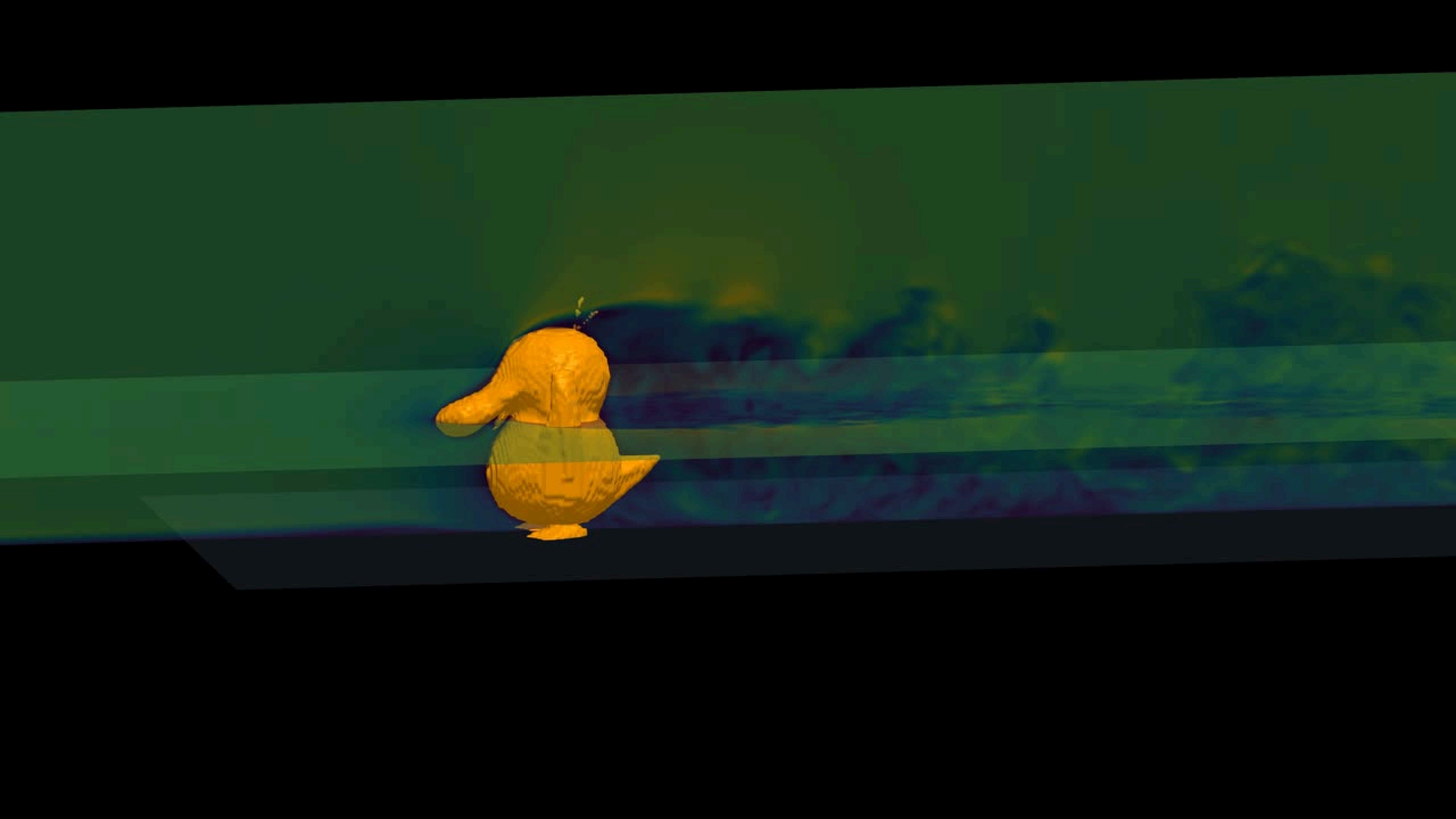
Psyduck 고라파덕 GPU domain-split visualization
(multi-GPU parallel processing context)



Psyduck 고라파덕 simulated wind-tunnel result (y = Ly/2 cross-section)
— y-plane velocity distribution + vector visualization



Psyduck 고라파덕 simulated wind-tunnel result (3D streamlines)
— 3D streamline visualization



7.

PIPELINE EXTENSION

Specifying Physical Conditions

Grid size settings

`--pitch / --target-cells`

Set directly, or auto-derive from a target cell count

Real-world size settings

`--target-height --scale-axis`

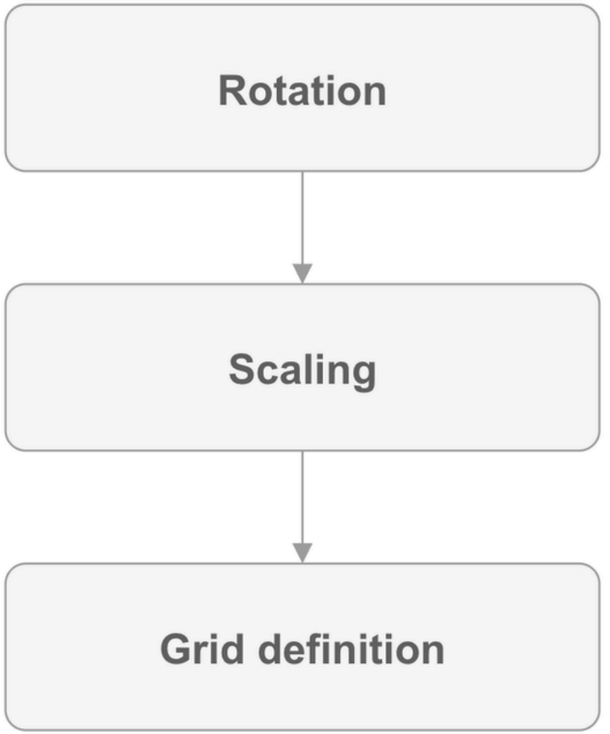
Uniformly scale to match real physical size (m)

Orientation settings

`--facing / --rotate-x/y/z`

Preset rotation, or specify rotation angles directly

Fixed application order

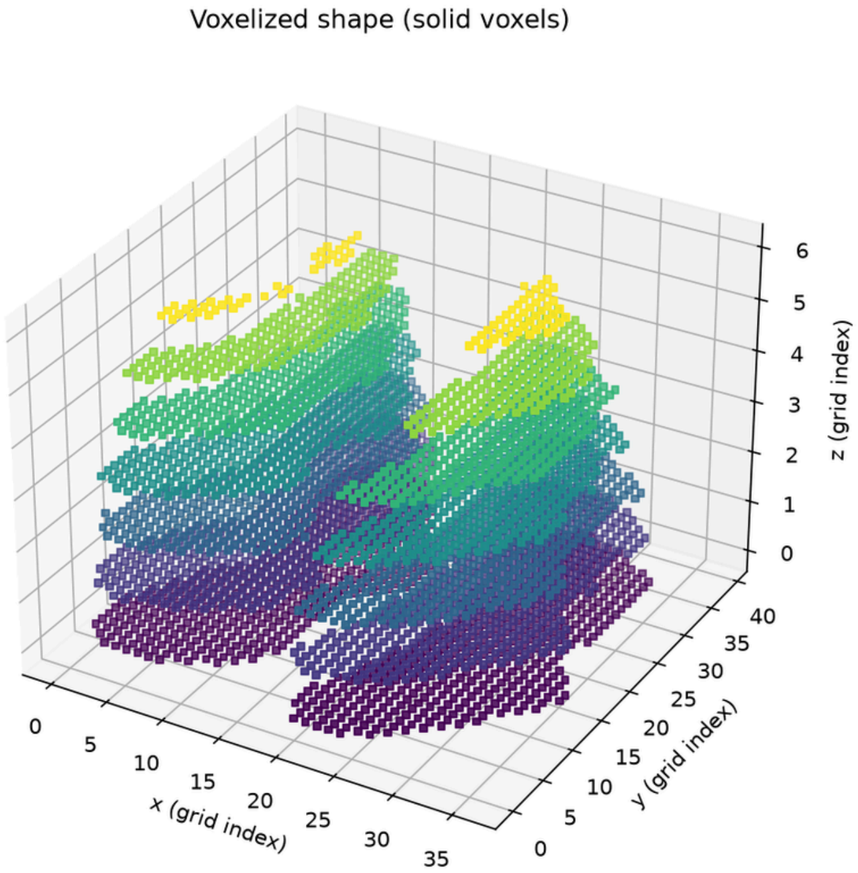


The bounding box must accurately reflect the state after each step

Validation — NASA Eros STL

Applying three features simultaneously:

- Rotated in the –Y direction (--facing negy)
- Scaled to 34,000 units along the x-axis
- Target grid: 40 cells (--target-cells 40)



33.15%
solid voxel

(37,40,8)
grid shape

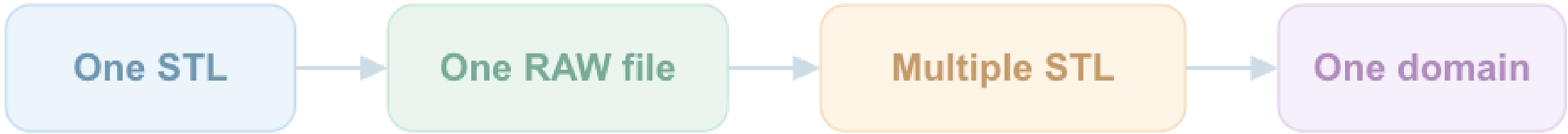
Results match expectations

8.

PLACING MULTIPLE BUILDINGS IN ONE DOMAIN

Background and Design

Design

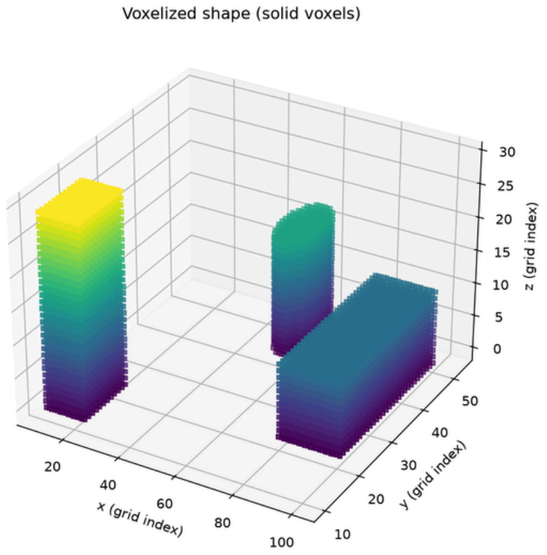


The final step completing the goal of "supplementing structures missing from OSM with STL"

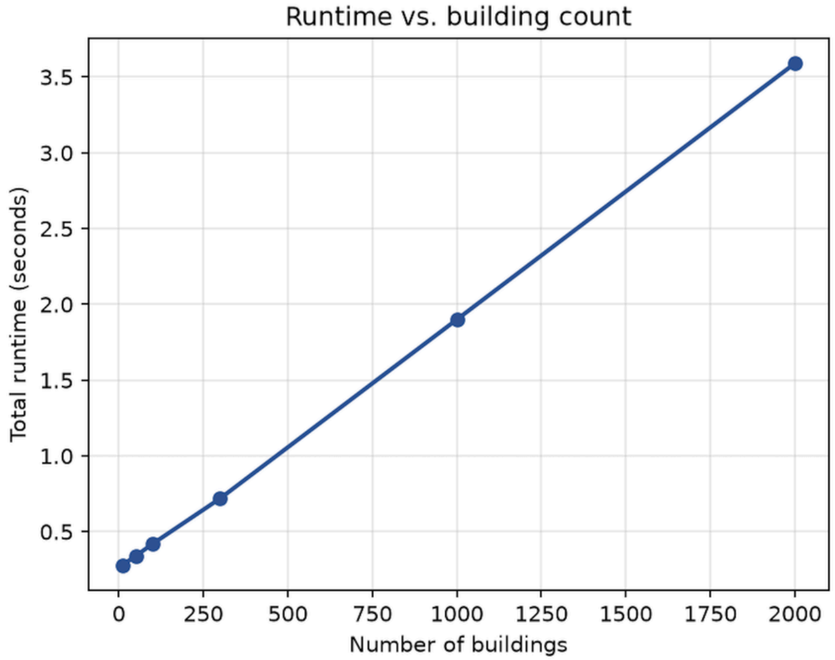
Design (5 steps)

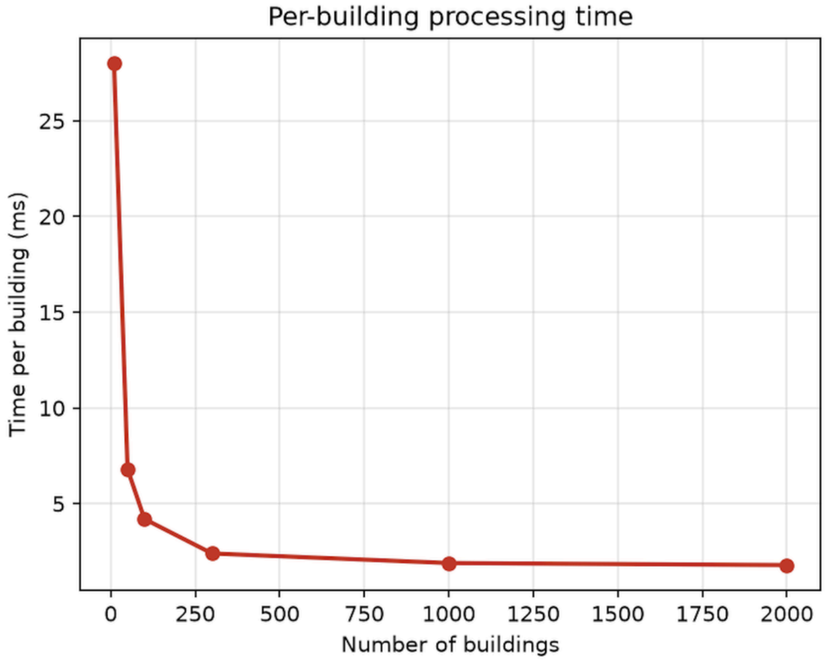
1. Explicitly specify domain size (nx, ny, nz) and pitch
2. Manage per-building placement info via a JSON config file
3. Rotate → scale → place at (x, y) coordinates on a ground-level basis
4. Each building computes only its own bounding-box region
5. Combine results from multiple buildings via an OR operation

Basic Behavior and Scalability Validation



3 buildings (rectangular prism, cylinder, rotated wide building); scalability test — 10 buildings → 2,000 buildings





Scalability test

10 → 2,000
buildings

0.28s → 3.52s
processing time

200× buildings → only ~12.6× time
sub-linear scaling

Solid voxel ratio

16,164
/ 480,000 voxels

3.37%
solid voxel

Round-trip validation

Passed

Remaining limitation: No feature yet to detect or flag overlap between buildings (handled safely via the OR operation)

```
# Domain index range this building's bounding box overlaps.
xr = index_range(x_centers, bmin[0], bmax[0])
yr = index_range(y_centers, bmin[1], bmax[1])
zr = index_range(z_centers, bmin[2], bmax[2])


inside = mesh.contains(points)
sub_occupancy = inside.reshape(len(sub_x), len(sub_y), len(sub_z))
slice_ = domain_occupancy[ix0:ix1, iy0:iy1, iz0:iz1]
# OR-combine into the shared domain array
domain_occupancy[ix0:ix1, iy0:iy1, iz0:iz1] = np.maximum(
    slice_, sub_occupancy.astype(np.uint8)
)
```

Closing

Background → Pipeline implementation → Validation → Real-world problem solving

Solver integration → Extended features → Multi-building placement

"Supplementing structures missing from OSM with STL"

is substantially completed with the multi-building placement feature

Remaining limitation

Overlap detection between buildings is not yet implemented

THANK YOU